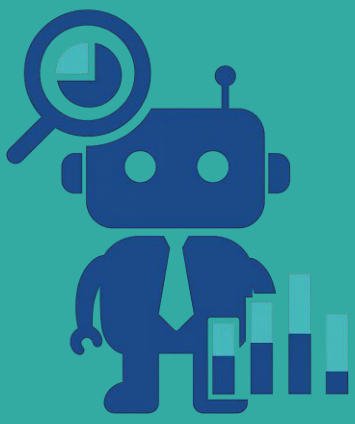
Classification supervisée et non supervisée pour traitement de données avec Orange

Projet : Novembre 2025





SCIENCES DE DONNEE : MME KHIARI



L2 - MIASHS
Créé par : HAKI Moatacem | 22408182
EJEBLI Adem | 22407392

Premiers pas

. Résumé de notre projet :

Ce rapport présente en détail les différentes étapes de traitement de données sur le logiciel Orange. Le projet s'oriente dans l'étude des méthodes de traitement automatique du langage appliquées à des **textes** (genre et synopsis d'anime), et de **données tabulaire** (information sur les joueurs de foot).
Il vise à confronter deux approches complémentaires : la **classification supervisée**, qui exploite des étiquettes de genre pour apprendre des règles de décision, et l'**analyse non supervisée**, qui cherche des structures et regroupements intrinsèques sans recours aux étiquettes.
Dans le cadre pédagogique de ce module, nous travaillerons sur **deux** jeux de données triées et optimisé pour nos recherches, en explicitant les procédés utilisés tout au long de la **chaîne de traitement**, puis nous **analyserons** les limites rencontrées par les modèles lors de la classification.

Table des Matières :

1. Apprentissage non supervisé _____ *3.*
1.1. Jeu de données _____ *3.*
1.2. Chaine de traitement _____ *3.*
1.3. Interprétation _____ *7.*
2. Apprentissage supervisé _____ *9.*
2.1. Jeu de données _____ *9.*
2.2. Chaine de traitement _____ *11.*
2.3. Interprétation _____ *15.*
3. Conclusion Générale _____ *18.*

1 – Apprentissage non supervisé

Introduction

Dans le cadre de ce projet, l'objectif est de construire et analyser deux chaînes d'apprentissage non supervisé à l'aide du logiciel **Orange**. Le jeu de données utilisé concerne des joueurs professionnels de football provenant de FIFA 20, un domaine que nous maitrisons, ce qui facilite l'interprétation des résultats. En effet, les statistiques des joueurs (vitesse, dribble, physique, défense, etc.) sont directement liées à leur profil réel sur un terrain, et permettent de comprendre comment les algorithmes regroupent les joueurs selon leurs caractéristiques.
Ce contexte rend l'analyse particulièrement pertinente : connaître le football et l'univers FIFA permet d'interpréter plus facilement les clusters, de reconnaitre les profils typiques de joueurs, et d'expliquer les regroupements obtenus par les méthodes de clustering.

1.1 – Présentation et préparation du jeu de données

Le jeu de données initial, issu de FIFA 20, comporte des informations très variées sur les joueurs : données personnelles (nom, âge, nationalité), caractéristiques physiques (taille, poids), statistiques de jeu (dribble, vitesse, défense, tir, physique), ainsi que des informations textuelles comme la ligue ou le poste occupé.

Cependant, les algorithmes de clustering utilisés en apprentissage non supervisé reposent sur des distances mathématiques entre individus, ce qui impose d'utiliser uniquement des variables numériques. Une étape de **nettoyage** du fichier CSV a donc été réalisée avant l'importation dans Orange.

Dans cette étape, toutes les colonnes textuelles ont été supprimées : nom du joueur, nationalité, équipe, poste, ligue, etc.

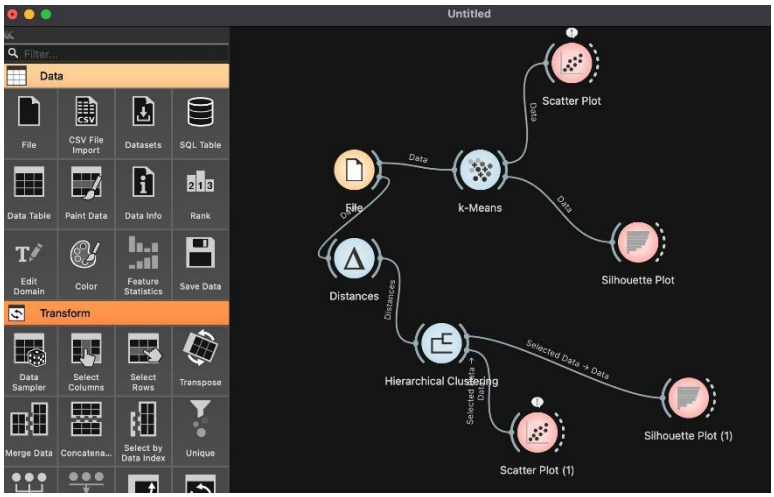
Seules les variables strictement numériques suivantes ont été conservées :
l'âge, la taille, le poids, la note générale, la valeur en euros, les gestes techniques, la vitesse, le tir, la passe, le dribble, la défense et le physique.

Après nettoyage, le jeu de données final contient **248 joueurs décrits par 12 variables numériques**. Cette préparation assure la compatibilité complète avec les méthodes de clustering et évite toute ambiguïté liée aux colonnes Meta ou Target dans Orange.

1.2 – Mise en place de la chaîne de traitement dans Orange

La chaîne de traitement a été construite conformément aux pratiques présentées en travaux pratiques. Le fichier nettoyé a été importé via le widget **File**, puis relié à deux approches distinctes de clustering : l'algorithme **K-Means** et le clustering hiérarchique. Une représentation visuelle complète de la chaîne est présentée ci-dessous.

Figure 1 – Chaîne de traitement complète dans Orange Clustering K-Means



Après importation, un premier clustering a été réalisé à l'aide du widget **K-Means**. Les variables ont été normalisées pour éviter que des colonnes sur des échelles très élevées (comme la valeur marchande) dominent artificiellement la mesure de distance.

Le nombre de clusters a été exploré entre 2 et 8, et les scores de silhouette ont été observés pour déterminer le nombre optimal. La configuration du widget K-Means utilisée est présentée ci-dessous.

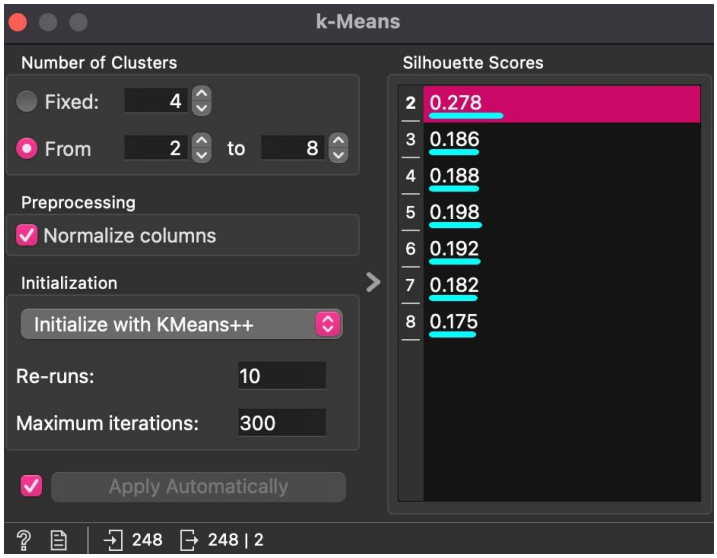


Figure 2 – Paramétrage du widget KMeans

Les résultats de silhouette obtenus montrent que la meilleure valeur est atteinte pour k = 2, avec un score d'environ 0,278. Le graphique correspondant est présenté ci-après.

Figure 3 – Silhouette Plot du K-Means

Une projection en deux dimensions à l'aide du Scatter Plot a ensuite permis d'observer la séparation des deux groupes. Les joueurs du premier cluster semblent davantage orientés vers les statistiques techniques, tandis que le deuxième regroupe des profils plus physiques ou défensifs.

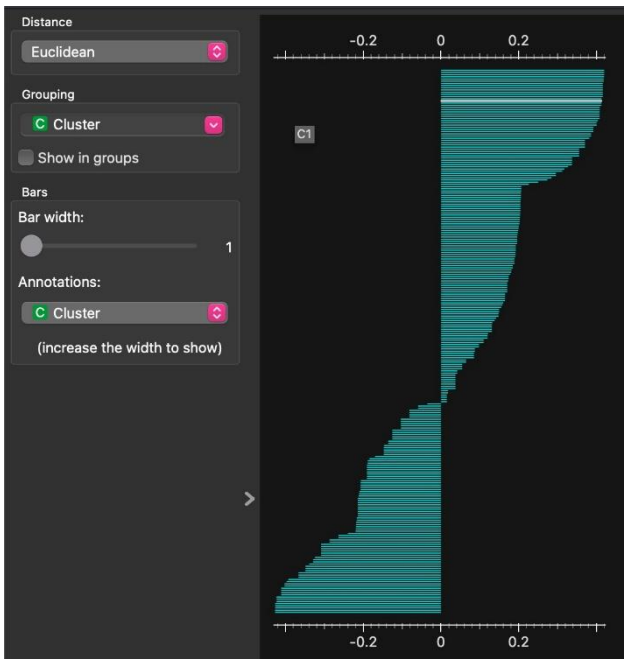
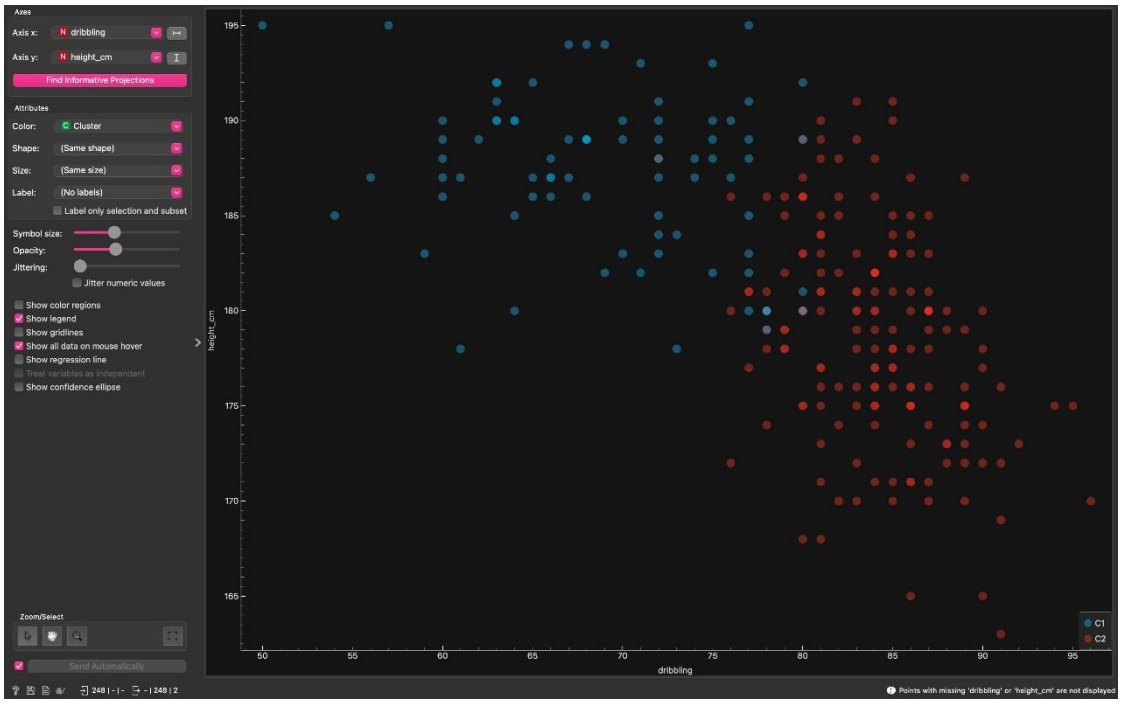


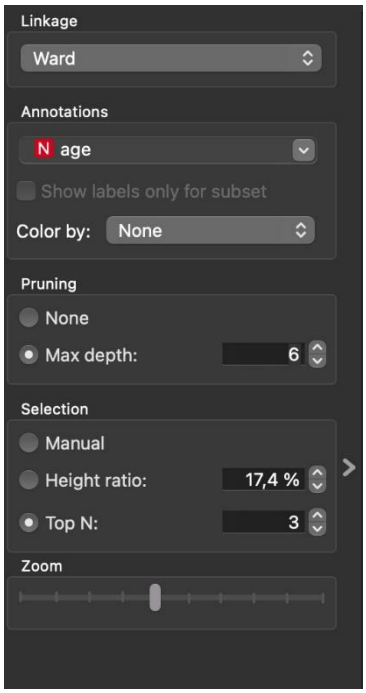
Figure 4 – Scatter Plot du clustering K-Means



Clustering hiérarchique

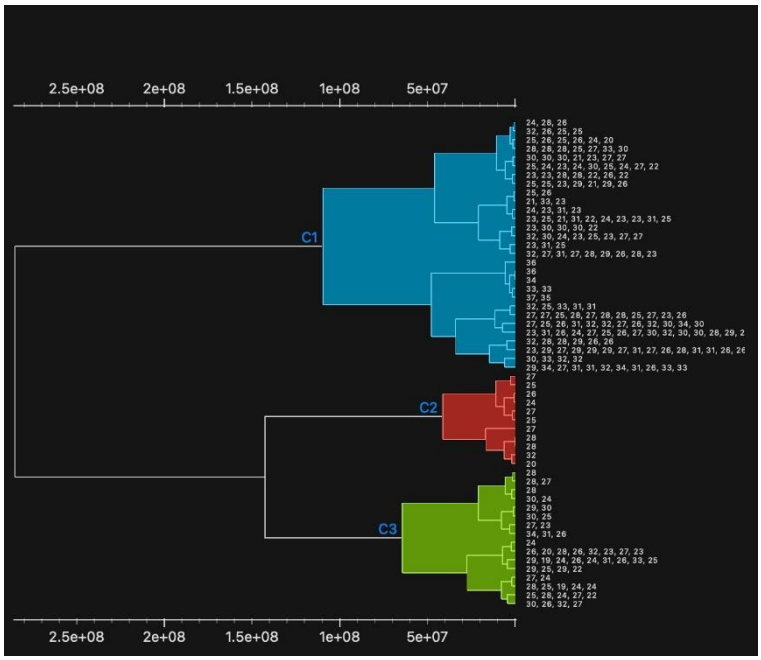
Une seconde méthode a été employée afin de comparer les résultats obtenus avec une approche différente. Le widget **Distances** a été utilisé avec la distance euclidienne, puis relié au widget **Hierarchical Clustering** configuré avec la méthode de Ward, comme recommandé dans les TP, car elle cherche à minimiser la variance intra-cluster.
La configuration utilisée est illustrée ci-dessous.

Figure 5 – Paramétrage du clustering hiérarchique (Ward, Top N = 3)



Le dendrogramme obtenu met en évidence une structure à **trois clusters,** ce qui semble plus cohérent que la séparation forcée en deux groupes observés avec K-Means.

Figure 6 – Dendrogramme du clustering hiérarchique



L'évaluation du clustering hiérarchique à l'aide du silhouette plot montre des valeurs nettement plus élevées, majoritairement comprises entre **0,3** et **0,6**, ce qui indique une bien meilleure qualité de séparation.

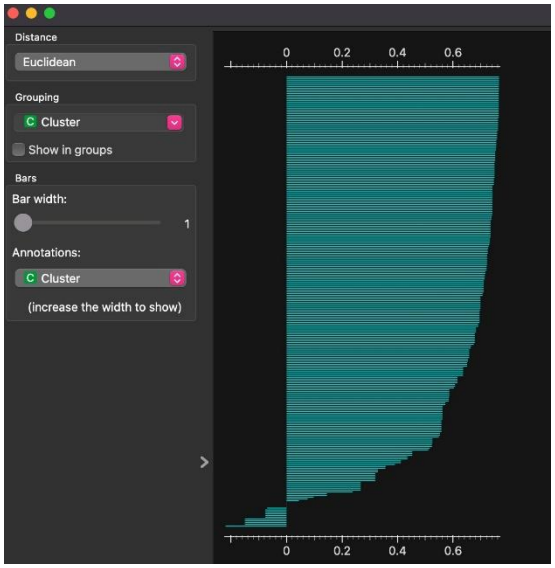
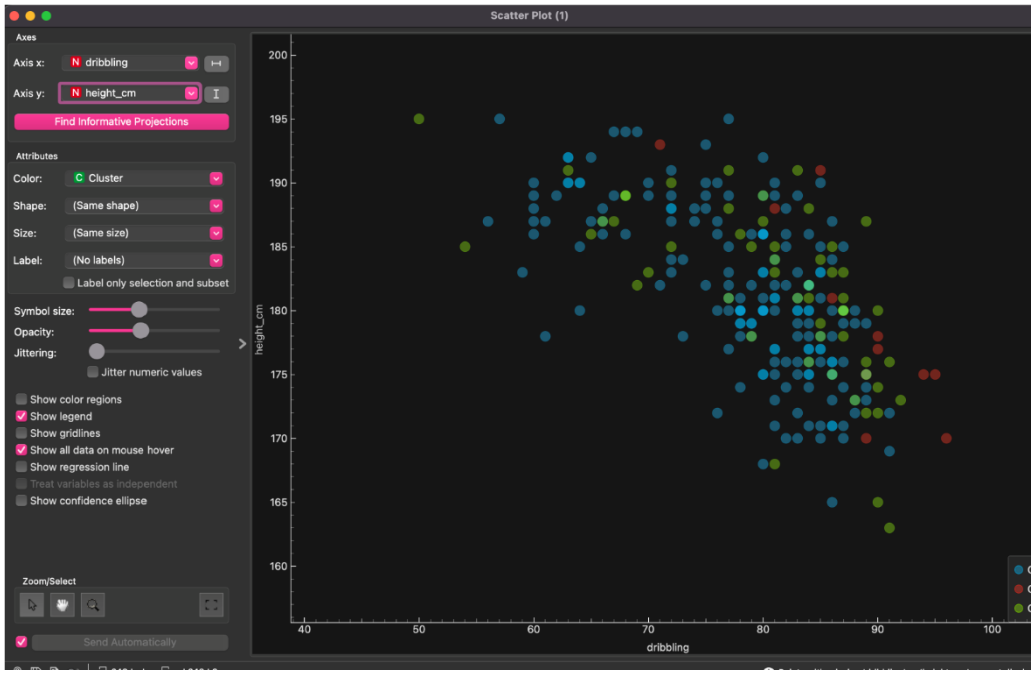


Figure 7 – Silhouette Plot du clustering hiérarchique

Enfin, un Scatter Plot à trois couleurs permet de visualiser, dans un espace réduit à deux dimensions, la répartition des clusters obtenus.

Figure 8 – Scatter Plot du clustering hiérarchique (3 clusters)



1.3 – Interprétation des résultats

L'analyse non supervisée a d'abord été menée avec l'algorithme K-Means. Le coefficient de silhouette calculé pour différentes valeurs de *k* montre que la meilleure partition obtenue correspond à *k = 2*, avec un score maximal d'environ **0,278** (voir *Figure 3 – Silhouette du K-Means*). Un score aussi faible traduit une séparation partielle des individus : les clusters sont présents, mais leur cohésion interne reste limitée. Le silhouette plot révèle en effet une proportion importante de silhouettes proches de zéro, ainsi que plusieurs valeurs négatives. Ces silhouettes négatives indiquent que certains joueurs sont plus proches du centre d'un autre cluster que du leur, ce qui reflète l'existence de profils hybrides difficiles à regrouper. Le scatter plot issu du **K-Means** confirme cette observation : bien que deux groupes émergent, une proportion notable de joueurs se situe dans une zone intermédiaire difficile à séparer visuellement.

Afin d'obtenir une segmentation potentiellement plus fidèle, un deuxième algorithme a été appliqué : le **clustering hiérarchique** (méthode de Ward). Le dendrogramme obtenu suggère naturellement une division en **trois clusters**, ce qui correspond à une structure plus riche que celle imposée par K-Means (voir *Figure 6 – Dendrogramme*). Le silhouette plot associé au clustering hiérarchique présente des résultats nettement plus satisfaisants, avec des silhouettes largement positives et atteignant des valeurs comprises entre **0,4** et **0,6** (voir *Figure 7 – Silhouette du clustering hiérarchique*). Contrairement à **K-Means**, on observe ici très peu de silhouettes négatives. Cela signifie que les individus sont mieux regroupés, que la cohésion interne des clusters est plus forte, et que la frontière entre clusters est globalement plus nette.

La comparaison directe entre les deux silhouettes montre donc une différence fondamentale :

— **K-Means** impose deux groupes qui ne reflètent pas entièrement la diversité des profils présents dans les données. Sa silhouette faible révèle l'existence d'un groupe intermédiaire que l'algorithme n'arrive pas à isoler.

— **Le clustering hiérarchique**, au contraire, identifie spontanément ce troisième groupe intermédiaire, ce qui améliore nettement la qualité de la partition. Le silhouette plot hiérarchique témoigne de cette cohérence supérieure.

L'interprétation des **clusters hiérarchiques** confirme cette analyse. Le premier cluster regroupe principalement des joueurs très techniques, caractérisés par un dribble, une vitesse ou une qualité de passe élevés. Le deuxième groupe rassemble des joueurs physiques ou défensifs, présentant une grande taille, un poids plus élevé ou des statistiques défensives supérieures. Enfin, le troisième cluster correspond à un groupe "intermédiaire", composé de joueurs équilibrés, ni extrêmement techniques ni particulièrement physiques. Ce groupe, absent dans le modèle **K-Means**, explique pourquoi ce dernier produisait un coefficient de silhouette faible : il tentait de forcer ces joueurs polyvalents dans l'un des deux clusters, alors qu'ils forment en réalité un groupe distinct.

Enfin, il est important de noter que les **scatter plots** 2D peuvent parfois donner une impression de mélange, notamment entre le cluster intermédiaire et l'un des deux autres groupes. Ceci est dû au fait que la projection en deux dimensions réduit une information initialement présente dans un espace beaucoup plus riche (plus de dix variables). Les valeurs du silhouette plot, calculées dans l'espace complet des données, constituent donc un indicateur beaucoup plus fiable de la qualité réelle du regroupement.

En conclusion, la comparaison des deux méthodes montre clairement que le clustering hiérarchique constitue une solution plus adaptée au jeu de données FIFA 20. Cette méthode offre une segmentation plus cohérente et met en évidence la présence d'un groupe intermédiaire que **K-Means** ne détecte pas. L'étude conjointe des silhouettes et des représentations graphiques permet ainsi d'obtenir une compréhension plus fine de la structure des données et des profils de joueurs.

2. Apprentissage supervisé

2.1 – Jeu de Données :

Pour l'apprentissage supervisé , nous avons choisi un jeu de données complet basée sur le traitement de texte ; disponible sur le site Kaggles , nous avons pris ce jeu de données au format csv contenant des information relative a plus de 60 000 titres d'anime différents (= film d'animation japonais) , nous avons du donc conserver uniquement les information pertinentes et réduire considérablement le nombre d'éléments .

Après nettoyage du fichier csv voici le résultat obtenu :

	A	B	C
1	titre	genre	synopsis
2	Naruto	Shonen	Naruto Uzumaki is a mischievous c
3	My Hero Academia	Shonen	In a society where most people pos
4	Monster	Seinen	Dr. Kenzo Tenma, a brilliant neuros
5	Psycho-Pass	Seinen	In a futuristic society governed by t
6	Fruits Basket	Shoujo	Tohru Honda, a kind-hearted high s
7	Kimi ni Todoke	Shoujo	Sawako Kuronuma is a shy girl misu
8	Doraemon	Kodomo	Nobita Nobi is a clumsy, unlucky bo
9	Pokémon	Kodomo	Ash Ketchum is a young boy who dr
10	Hamtaro	Kodomo	Hamtaro is a curious hamster who
11	Nana	Josei	Two women named Nana meet on a
12	Usagi Drop	Josei	Daikichi, a thirty-year-old bachelor
13	Ergo Proxy	Seinen	In a domed, post-apocalyptic city,
14	One Piece	Shonen	Monkey D. Luffy, a young pirate wit

Légende des variables :

Titre : cette variable contient des titres sous forme de chaine de caractère, elle Permet de **différencier** efficacement les données.

Genre : cette variable est la **variable cible** a 5 valeurs possibles : Shonen / Seinen / Shoujo / Kodomo / Josei. Nous expliquerons leur valeur par la Suite.

Synopsis : C'est sur cette variable textuelle que ce baserons les modèles pour Prédire le genre.

Genre :

Nous avons utilisé les catégorisations japonaises car elles englobent plusieurs catégorie « françaises » à la fois et permettent donc de classer plus efficacement les éléments sans trop augmenter les valeurs possibles de la variable, voici leur signification :

Shonen : Action / Aventure / Camaraderie. *Plutôt destiné **à tout public***
Seinen : Réalisme / Mystère / Psychologie. *Plutôt destiné à un **public mature***
Shoujo : Romance / Vie Quotidienne / Emotions. *Plutôt destiné à **tout public***
Josei : Romance / Drame/ Vie Professionnelle. *Plutôt destiné à un **public mature***
Kodomo : Enfants / Humour / Education. *Plutôt destiné à un **jeune public***

Synopsis :

Ils sont tout simplement une description de l'œuvre a laquelle ils sont liées, ils sont en anglais et sont donc plutôt long (environs 50 – 65 mot) , les modèles déduirons le genre selon cette variable.

Sous **recommandations de la professeure**, il est préférable de travailler sur **effectif restreint** (train : 10 , test : 3) , ou à plus grande échelle (train : 20 , test : 5) ; nous avons donc directement réduit les effectifs de notre jeu de données . Liens pour la base de données complète : **MyAnimeList Anime and Manga Datasets**

→ Ici travailler sur un effectif restreint permet de mieux comprendre et interpréter les résultats obtenus par nos chaines de traitements ; et l'échantillons un peu plus grand permet d'obtenir des résultats plus significatifs.
Finalement il y a donc **13 / 25 lignes** dans notre jeu de données.

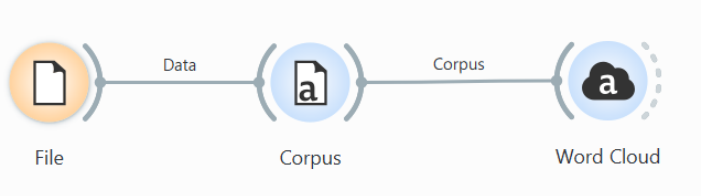
Passons maintenant à l'étape clé de notre rapport : le traitement du jeu de données , ou nous allons l'exploiter et en tirer des résultats puis des conclusions

2.2 – Chaine de traitement :

Commençons par ouvrir Orange , dans le widget File nous allons directement définir la variable genre comme Target (variable cible) **.**

	Name	Type	Role	Values
1	genre	categorical	target	Josei, Kodomo, Seinen, Shonen, Shoujo
2	titre	text	meta	
3	synopsis	text	meta	

Une fois tout les rôles attribué nous vérifions dans un premier temps que les données ont été correctement ouvertes via le widget ***Data Table***, comme tout est correct nous pouvons maintenant créer la suite : nous devons dans un premier temps rendre les valeurs textuelles contenues dans la colonne synopsis accessibles pour qu'ils puissent être traité. Pour cela nous relions l'outils corpus a la sortie de ***File***, puis nous visualisons nos données avec le ***Word Cloud***

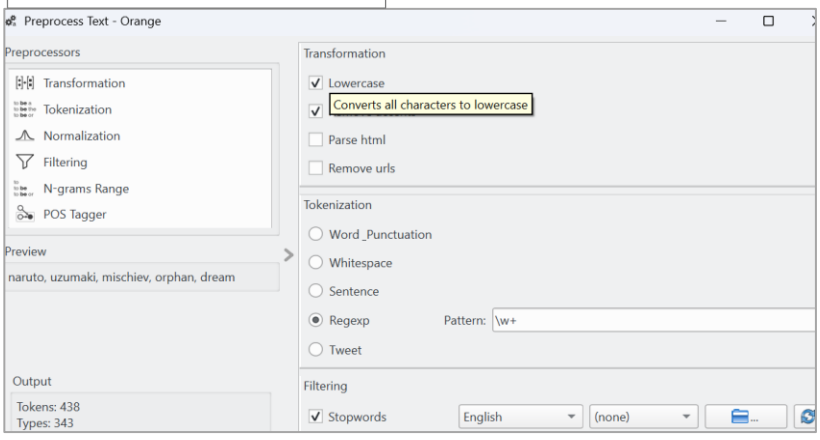


Vue du Word Cloud :



Comme nous pouvons le constater beaucoup de mot dont le traitement ne nous intéresse pas et qui peuvent perturber nos modèles (the , of , a …) ; nous allons donc trier les mots avec le widget ***Preprocess Text*** en sortie de Corpus, après l'application des réglages suivants, nous obtenons un résultat plus satisfaisant (nouveau ***Word Cloud*** a la sortie de ***Preprocess Text***).

Vue du Preprocess Text :

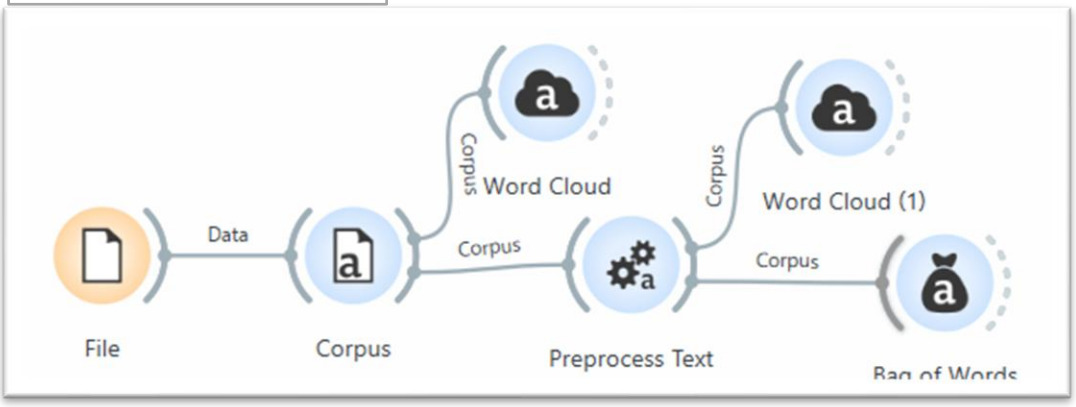


Vue du 2nd Word Cloud :



Ajoutons maintenant un ***Bag of Word*** en sortie de ***Preprocess Text*** et nos données sont maintenant exploitables !

Vue de la chaine actuelle :

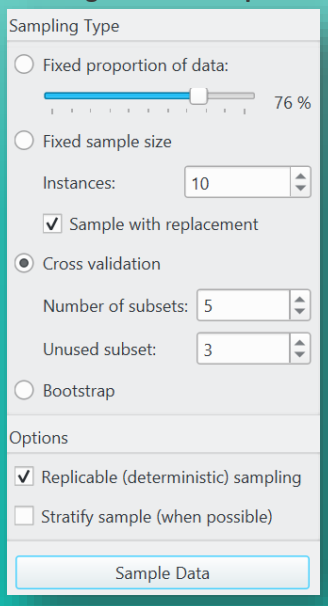


Désormais le prochain objectif est de **tester** différents modèles pour la classification de nos données, mais d'abord divisions les en 2 parties, 10 enregistrements servirons a la **phase d'entrainements**, tandis que les 3 autres seront utilisé lors de la **phase de Test**, pour cela nous allons ajouter le widget Data Sampler qui se chargera de la divisions de nos données avec la configuration suivante :



Data Sampler

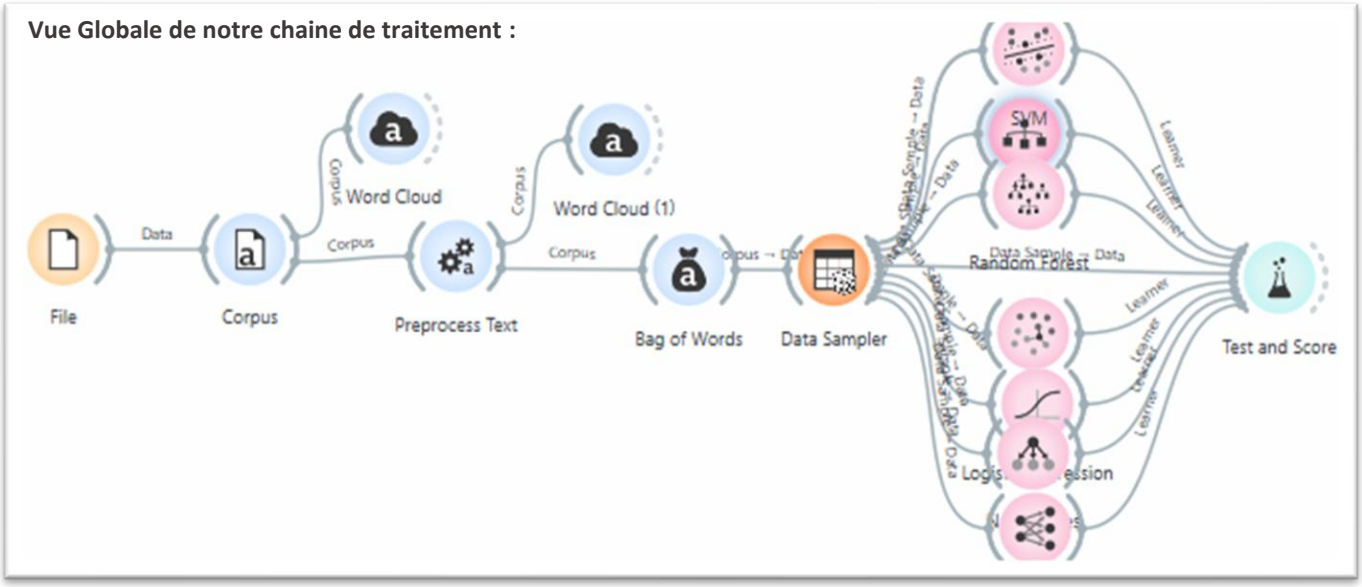
Config du Data Sampler :



Puis nous lions ce ***data sampler*** a tous les modèles disponibles afin de les entrainer, cela nous permettra notamment de voir les modèles qui perforent le mieux pour la classification dans notre jeu de donnée.

Nous avons donc la liste des 7 **modèles** (ici ***SVM***, ***KNN***, ***Neural Network***, ***Logistic Regression***, ***Naive Bayes***, ***Tree*** et ***Random Forest***) relié directement au ***data Sampler*** en entrée (c'est lui qui leur fournit les enregistrement) et ***Test and Score*** en Sortie pour tester nos **Modèles** .

Vue Globale de notre chaine de traitement :



Voici a droite les resultats de nos modèles, nous pouvons voir que 3 modèle on fait un **score parfaits** , mais les resultat qui nous interressent sont les resultats de test que nous allons mettre en place juste après cette partie .

Model	AUC	CA	F1	Prec	Recall	MCC
SVM	0.000	0.300	0.138	0.090	0.300	0.000
Random Forest	1.000	0.700	0.585	0.513	0.700	0.647
Tree	0.974	0.900	0.860	0.833	0.900	0.882
kNN	0.833	0.700	0.627	0.583	0.700	0.645
Logistic Regression	1.000	1.000	1.000	1.000	1.000	1.000
Neural Network	1.000	1.000	1.000	1.000	1.000	1.000
Naive Bayes	1.000	1.000	1.000	1.000	1.000	1.000

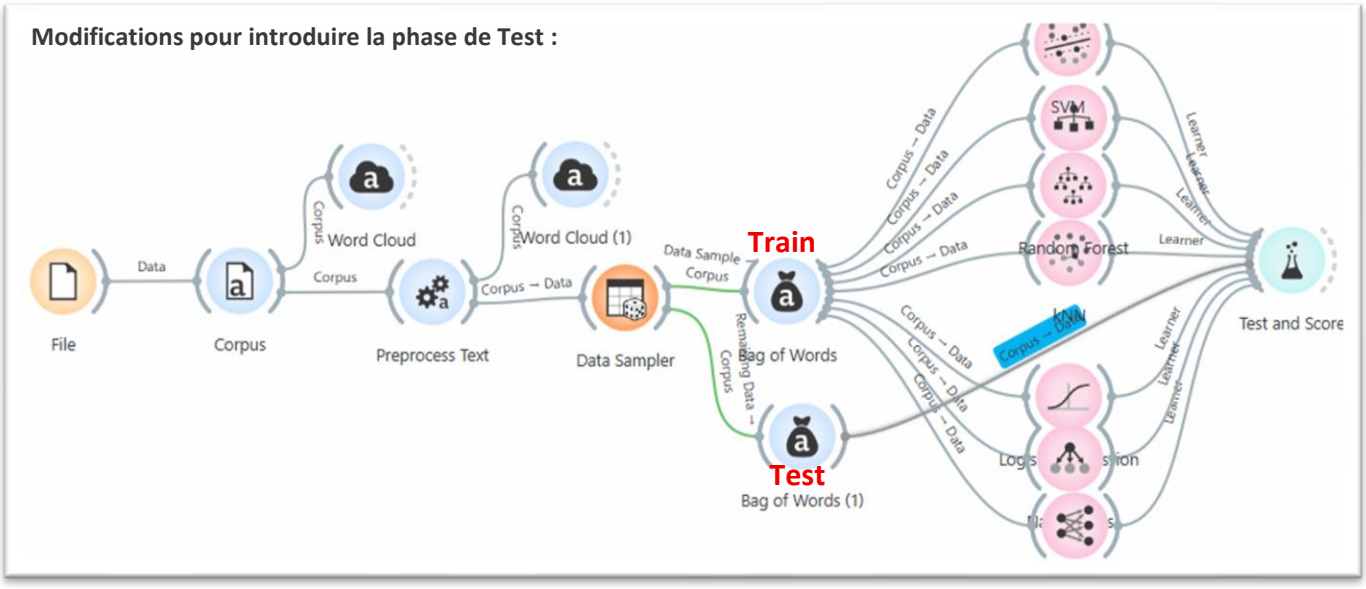
Mise en place de la phase de test :

A présent nous allons mettre en place la **phase de Test**, pour cela quelques modifications doivent être apporté a notre chaine de traitement ; en effet nous allons relier 2 ***Bag of Words*** à la sortie de notre ***Data Sampler*** *(et retirer celui a son entrée)*:

Le premier sera dédié à la **phase d'entrainement**, il contient donc **80%** des données et sera donc liée uniquement aux différents modèles d'apprentissage.

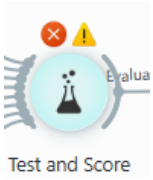
Le second sera dédié à la phase de Test, ce ***Bag of Word*** vectorise donc les **20%** des données restantes et sera liée uniquement à ***Test and Score***.

Modifications pour introduire la phase de Test :



Commençons par rendre les résultats plus clair et lisibles nous allons ajouter une ***Confusion Matrix*** en sortie de ***Test and Score*** pour obtenir la **matrice de confusion**.

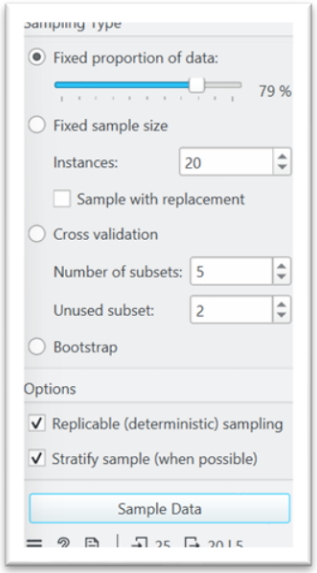
Cependant lors du Test nous avons rencontré le problème suivant, le modèle KNN ne pouvais pas fonctionner car le nombre d'enregistrement Test était inferieures aux nombre de valeur possible de la variable Genre (3 enregistrements pour 5 genre possible) :



kNN (error)

Expected n_neighbors <= n_samples_fit, but n_neighbors = 5, n_samples_fit = 3, n_samples = 3

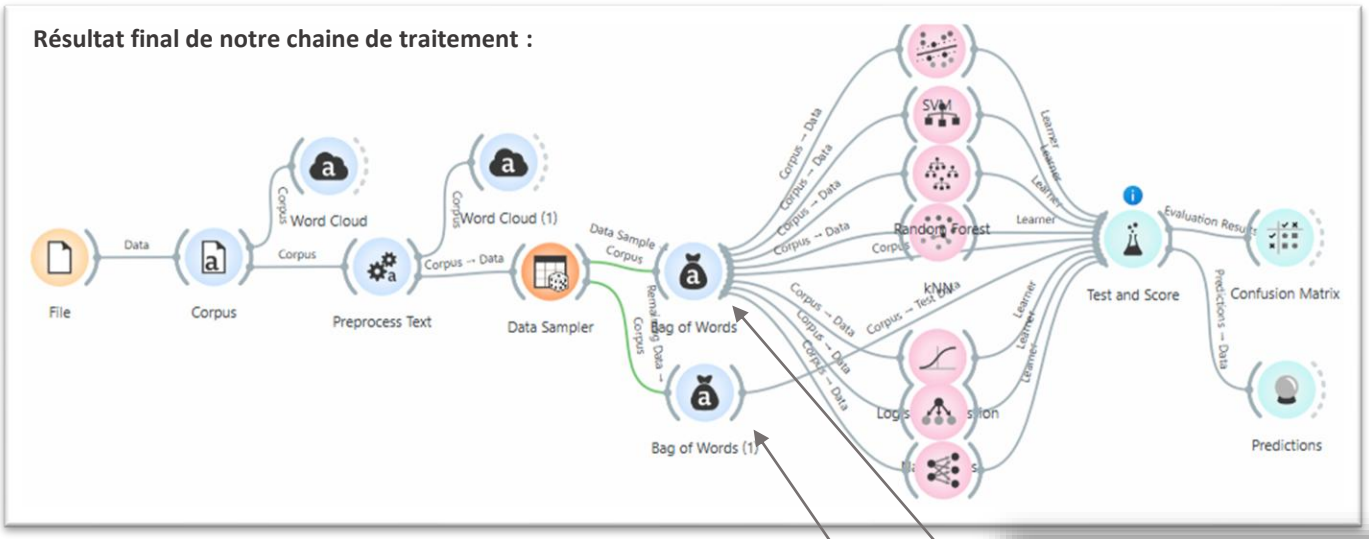
Nous avons donc pris un échantillons plus grand contenant à présent 5 enregistrement pour la phase de Test , et donc 20 pour la phase d'entrainement pour conserver les proportions 80 – 20 .

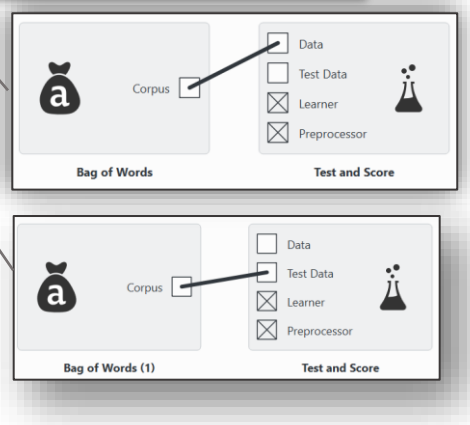


Voici donc les nouveaux paramètres de ***Data Sampler*** pour repartir les données comme mentionné précédemment.

Enfin nous connectons l'outils **Prédictions** pour pouvoir **visualiser** plus facilement l'ensembles des scores simultanément , cela nous servira pour la dernière partie .

Résultat final de notre chaine de traitement :





Note : *Il est important de bien câbler différemment le* ***Bag of Words*** *du* ***test*** *de celui dédié a* ***l'entrainement*** *comme le montre le schéma ci-contre*.

Nous avons donc montrez le procédé de la création de notre chaine de traitement et pouvons à présent entamer la dernière partie de ce second point sur la classification supervisé.

2.3 – Interprétation des résultats

Dans cette dernière sous partie nous allons expliciter les résultats et en tirer des conclusions, nous verrons aussi que chaque modèle a ses domaines d'application et que certain modèle voit leurs limites atteintes.

Tout d'abord observons de plus près les résultats de ***Test and Score***, sans grande surprise le modèle ***Logistic Regression*** est globalement le plus **performant**, mais nous pouvons voir que les résultats diffèrent bien de ceux vu lors de la **phase d'entrainement**, pour la simple et bonne raison que nos **modèles** n'ont jamais vu les données de Test auparavant, il est donc difficile de faire un score parfait. Voici les statistiques de performance des **modèles** :

Statistiques relatives aux différents modèles :

Model	AUC	CA	F1	Prec	Recall	MCC
SVM	0.850	0.600	0.467	0.400	0.600	0.559
Random Forest	0.825	0.400	0.267	0.200	0.400	0.280
Tree	0.775	0.600	0.500	0.467	0.600	0.598
kNN	0.775	0.600	0.533	0.500	0.600	0.527
Logistic Regression	1.000	0.800	0.733	0.700	0.800	0.791
Neural Network	0.700	0.600	0.533	0.500	0.600	0.527
Naive Bayes	0.950	0.600	0.500	0.467	0.600	0.598

Même sur chaque point isolé , Logistic Regression reste le meilleur , nous pouvons voir que les autres modèles ce concurrence pas mal , excepté Random Forest qui est globalement le moins performant mais conserve tout de même un très bon score concernant les Air Sous courbes (AUC) .

Random Forest :

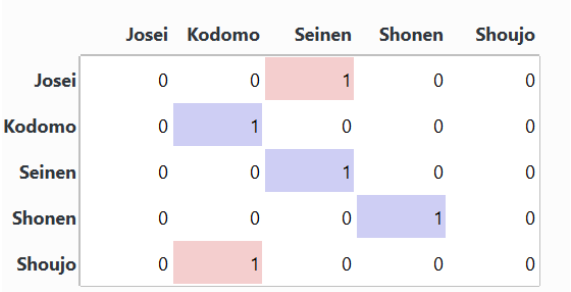
	Josei	Kodomo	Seinen	Shonen	Shoujo
Josei	0	1	0	0	0
Kodomo	0	1	0	0	0
Seinen	0	0	1	0	0
Shonen	0	0	1	0	0
Shoujo	0	0	0	1	0

Le modèle Random Forest montre une performance fragile avec **40 % de bonnes prédictions**. Il distingue correctement les genres Kodomo, Seinen, mais confond Josei avec Kodomo et Shonen avec Seinen. Ces erreurs suggèrent une sensibilité limitée aux nuances lexicales spécifiques à certains genres, probablement liée à la nature du modèle et à la représentation vectorielle utilisée.

L'erreur Josei → Kodomo pourrait s'expliquer par la présence de termes comme "enfant", "famille", ou "quotidien", qui sont fréquents dans les synopsis Kodomo. Le modèle, basé sur des arbres, a probablement surpondéré ces mots sans capter le ton mature du Josei.

Celle de Shonen → Seinen pourrais ce traduire par les thermes très « Masculin » employé dans les synopsis de ses 2 genre .

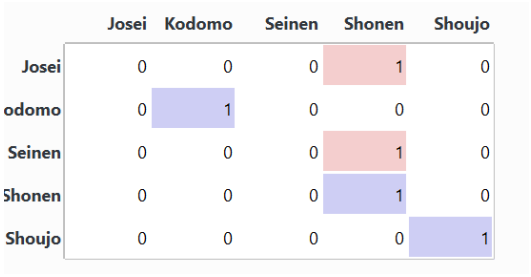
Tree / SVM :



	Josei	Kodomo	Seinen	Shonen	Shoujo
Josei	0	0	1	0	0
Kodomo	0	1	0	0	0
Seinen	0	0	1	0	0
Shonen	0	0	0	1	0
Shoujo	0	1	0	0	0

Les modèles Tree et SVM montrent une performance partielle, avec **60 % de bonnes prédictions**. Ils reconnaissent correctement les genres Kodomo, Seinen et Shonen, mais confondent Josei et Shoujo avec Kodomo. Ces erreurs suggèrent une difficulté à distinguer les genres plus subtils (Josei, Shoujo) lorsque le vocabulaire est partagé avec des œuvres destinées à un public plus jeune.
L'erreur Shoujo → Kodomo pourrait venir de la présence de termes comme "école", "jeune fille", "animaux mignons", qui sont aussi présents dans des synopsis Kodomo. Le modèle a probablement associé ces mots à une cible enfantine, ignorant les thèmes romantiques ou sociaux typiques du Shoujo.
L'erreur Josei → Seinen Les 2 genre utilisent un vocabulaire réaliste et légèrement dramatique/sombre, pour certains modèles assez tragiques cela pourrait créer une confusion.

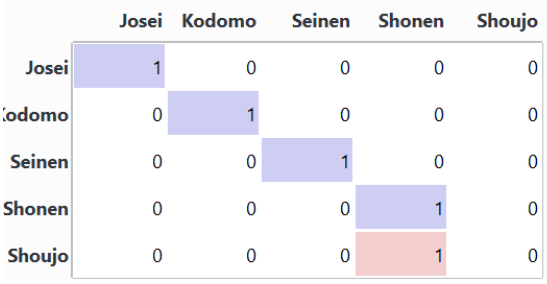
KNN / Neural Network / Naives Bayes:



	Josei	Kodomo	Seinen	Shonen	Shoujo
Josei	0	0	0	1	0
odomo	0	1	0	0	0
Seinen	0	0	0	1	0
Shonen	0	0	0	1	0
Shoujo	0	0	0	0	1

Les modèles **[Naive Bayes / kNN]** ont atteint une précision de **60 %,** avec des prédictions correctes pour les genres Kodomo, Shonen et Shoujo. Les erreurs concernent Josei et Seinen, tous deux **confondus** avec Shonen. Cela suggère une difficulté à distinguer les **genres matures** (Josei, Seinen) du genre Shonen, probablement en raison de la présence de vocabulaire partagé dans les synopsis (ex: "combat", "relation", "épreuve").

Logistic Regression:



	Josei	Kodomo	Seinen	Shonen	Shoujo
Josei	1	0	0	0	0
odomo	0	1	0	0	0
Seinen	0	0	1	0	0
Shonen	0	0	0	1	0
Shoujo	0	0	0	1	0

Le modèle **Logistic Regression** atteint une performance optimale **(80%)** sur l'échantillon testé, avec une classification très correct pour **l'ensemble** des genres. Cette matrice de confusion parfaitement diagonale témoigne d'une séparation claire des classes et d'une vectorisation efficace du texte. Le modèle parvient à capter les spécificités lexicales propres à chaque genre, même dans un corpus réduit .

Interprétation finale :

Finalement, bien que certains **modèles** aient atteint une classification parfaite, la majorité présente des confusions récurrentes entre genres **proches lexicalement**, comme Josei et Shonen. Ces erreurs soulignent les limites des approches basées uniquement sur la **fréquence des mots**, sans prise en compte du contexte sémantique. Pour améliorer la robustesse, l'intégration de techniques plus avancées ou l'augmentation du corpus pourrait affiner la **séparation entre genres**. Une analyse plus fine des mots discriminants et des regroupements vectoriels permettrait également de mieux comprendre les zones de confusion et d'optimiser les performances futures.

3. Conclusion

Ce projet a mis en évidence, sur deux contextes complémentaires , que la qualité d'une chaîne de traitement dépend autant de la préparation des données et des choix méthodologiques que des algorithmes eux-mêmes. Les résultats obtenus montrent une performance solide en **supervision** (notamment avec ***Logitsic Regression***) et une segmentation plus cohérente en **non-supervision** via le **clustering hiérarchique**, tout en révélant les limites structurelles des approches testées.

La mise en œuvre de ce projet nous a été particulièrement instructive : elle nous a fait passer de la théorie à la pratique sur de **vraies bases de données**, avec leurs contraintes de nettoyage, choix de variables et validation.

La rédaction de ce rapport, nous a initiés à certaines exigences : documenter la chaîne de traitement, justifier les choix, interpréter les résultats et assumer les limites des modèles.